24-hour-ahead electricity load forecasting: recurrent vs. convolutional networks and the role of seasonal baselines

Matthias Starck
github.com/mstarckgh/ele680

November 4, 2026

1 Introduction

Electricity load has strong daily and weekly cycles. A forecast for the next day can use information from up to a week back. Bai et al. [1] argue that generic temporal convolutional networks (TCN) match or beat recurrent networks on sequence tasks. Recurrent networks are for example the long short-term memory (LSTM) and the gated recurrent unit (GRU). Lai et al. [2] discuss why recurrent models struggle with long periodic patterns on this same dataset. I revisit this comparison on data from the UCI Machine Learning Repository [3] with a one-shot 24 h forecast. The aim is to replicate the comparison of sequential and convolutional networks qualitatively and to explain the results. My setup differs from both papers in the dataset subset, the forecast horizon and the metrics. I also ask whether the networks add anything beyond simple seasonal forecasts.

My contributions, in the order the work developed, are as follows.

1. A negative baseline result. All three networks lose to "repeat last week" (Section 3.1).
2. A residual formulation. The network predicts only a correction to a seasonal forecast. I compare two anchors, last week's load and the four-week mean (Sections 3.1, 3.4).
3. Stronger non-learned baselines (multi-week averages) and a paired evaluation over test days with block-bootstrap intervals. This changes several conclusions that the mean absolute error (MAE) alone would suggest (Sections 3.2–3.4).

2 Methods and materials

Data. The dataset provides 370 clients. I used 10 of these clients with no zero-load hours since 2012 to minimize preprocessing steps. The clients (MT_156, 159, 169, 232, 242, 248, 266, 281, 302, 307) were drawn once using a fixed seed. The 15-minute power readings in kW are averaged to hourly values in kWh. The dataset was split chronologically 70/10/20 into train/validation/test sets. Model windows never cross split boundaries. Validation and test windows start at midnight so every forecast covers one complete day. An overview can be seen in Figure 1.

Task and metrics. The models get the last L hours of all 10 clients ($L \times 10$) and output the next 24 hours of all clients in one shot (24×10). One model serves all clients. A linear layer maps the last hidden state (LSTM, GRU) or the last time step (TCN) to all 24×10 outputs. Each client is z-scored with the mean and standard deviation of its training data. The metrics MAE (mean absolute error) and RMSE (root mean square error) are computed in kW which is numerically the same as kWh since hourly values are predicted. I also used WAPE (weighted absolute percentage error). The clock change in October doubles the power consumption for one 1:00 hour. A secondary "noDST" MAE excludes the two affected days and changes the MAE at most 0.05 kW.

Models. Bai et al. [1] used models of roughly 70k parameters. I tried to match that for all the models I used, so that the capacity is comparable, and arrived at these models:

- LSTM with 2 layers $\times$ 64 units (68,336 parameters)
- GRU with 2 layers $\times$ 72 units (67,200 parameters)
- TCN with 7 residual blocks of two weight-normalised causal dilated convolutions, kernel 3, dilations $1, 2, \ldots, 64$ and 38 channels ($68,298$ parameters, receptive field $1 + 4(2^n - 1) = 509$ h)

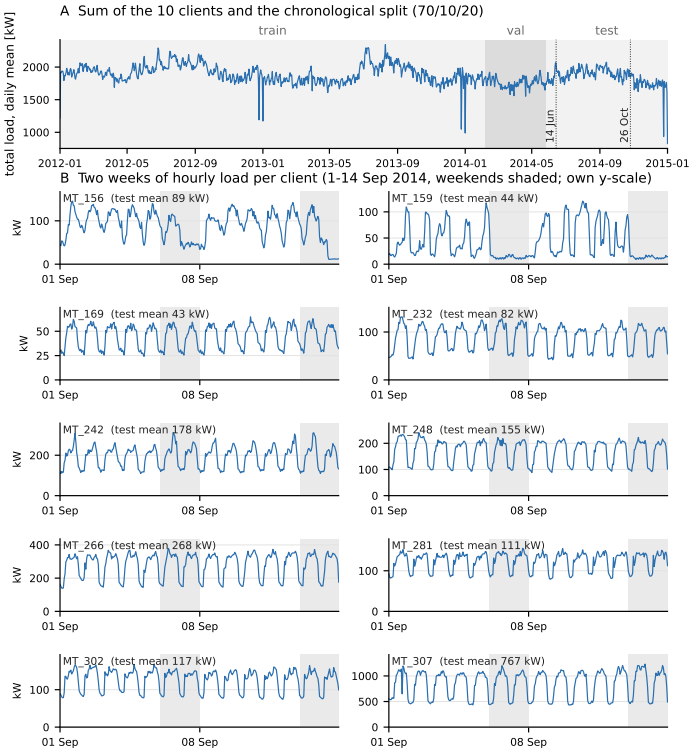


Figure 1: Overview of the selected clients and the periodic nature of the data

The models are implemented in PyTorch and follow [1]. I trained them with Adam (learning rate 10^{-3}), a batch size of 64, mean squared error (MSE) on normalised data, gradient clipping at 1.0, dropout 0.1, at most 40 epochs and early stopping with patience 6. Every configuration is trained with five different seeds. I also tried L1 loss (the mean absolute error as training objective) for the GRU and TCN. At $L = 168$ it changed the mean MAE by 0.4 kW or less. At $L = 672$ the residual TCN with L1 loss had a lower mean MAE than with MSE (13.52 against 14.02 kW). I ran no paired test, so I draw no conclusions. All tables use MSE.

Residual variants. The network output is added to a seasonal forecast from the input window. The network therefore only learns a correction. The forecast is either the same 24 hours one week earlier (needs $L \geq 168$) or their mean over the last four weeks (`residual: mean4`, needs $L \geq 672$). There is no residual version of the LSTM. It was the weaker of the two recurrent networks, so I ran the residual variants only for the GRU and the TCN.

Baselines. I used five baselines. Naive-24h repeats yesterday and naive-168h repeats the same day one week earlier. Mean-2w and mean-4w average the same 24 hours over the last two and four weeks, and median-4w takes the median over four weeks. These reduce the influence of a single unusual week. The four-week baselines need 672 h of history.

Paired evaluation. A window needs L hours of history inside the test split, so a model with input length L is scored on days that start L hours into the test period. $L = 672$ sees 21 fewer days than $L = 168$ (191 instead of 212). The 191 days start on 24 June, so 3 to 23 June is missing. Mean MAEs for different L therefore use different days, and I make all important comparisons on the days that both models cover. For a model and a reference I report the mean daily MAE difference (reference minus model, so positive favours the model, averaged over seeds). I add a 95% confidence interval (CI) from a moving-block bootstrap with 7-day blocks, because daily errors are autocorrelated. If the interval stays above zero, the model is probably better. If it contains zero, a tie is possible. I also report the share of days on which the model wins and the range over seeds. The intervals cover the sampling of days only, not seed variance, and only this single test period. I report many intervals, so a few that barely exclude zero are expected by chance. The paired runs were retrained with forecasts saved and can differ from the mean tables by about the seed spread.

Reproducibility. All configurations are generated by `scripts/make_configs.py`, and the `README.md` lists the commands to reproduce all results.

3 Results

3.1 Direct and residual forecasts

Table 1 shows the results for $L = 168$. Naive-24h has an MAE of 14.46 kW and naive-168h of 14.43 kW (RMSE 30.7 and 28.3). Every directly trained network is worse than naive-168h. The LSTM is worst, and GRU and TCN are indistinguishable. The gap is concentrated in client MT_159, whose mean load is 44 kW. Its WAPE is 45–55% for the direct models against 30% for naive-168h. The residual formulation lowers the MAE by 2.0 kW for the GRU and by 2.7 kW for the TCN. The GRU then reaches the level of naive-168h, and the TCN has a lower mean MAE by about 0.55 kW. Section 3.2 shows how much of the TCN's gain is reliable. Figure 2 shows the forecasts of all directly trained models and the mean-4w baseline for a typical weekday.

Table 1: Test MAE [kW], $L = 168$, mean $\pm$ standard deviation over 5 seeds.

	direct, MSE	residual, MSE
LSTM	17.87 ± 0.45	–
GRU	16.49 ± 0.25	14.49 ± 0.13
TCN	16.58 ± 0.34	13.87 ± 0.13

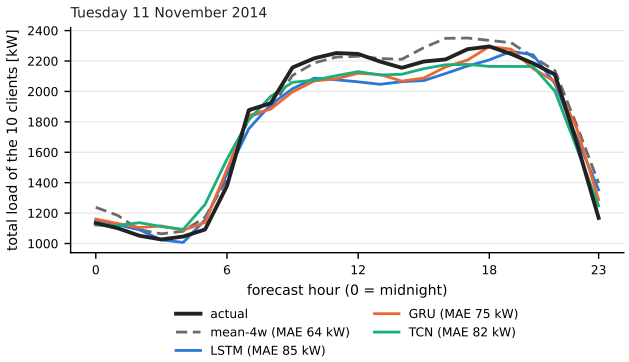


Figure 2: Forecasts of the directly trained LSTM, GRU and TCN and of the mean-4w baseline for a typical weekday (11 November 2014), summed over the 10 clients. The legend gives the MAE of the summed load on that day.

3.2 Paired comparison over test days

Table 2: Paired comparison on common test days. The column diff is reference MAE minus model MAE in kW, so positive favours the model. CI is the 95% moving-block bootstrap interval. The column better is the share of days on which the model wins.

model	reference	days	diff	95% CI	better	per-seed diff
GRU (res, L168)	naive-168h	212	−0.06	[−0.26, 0.18]	43%	−0.29 .. 0.11
TCN (res, L168)	naive-168h	212	0.46	[0.02, 1.07]	47%	0.28 .. 0.54
TCN (res, L168)	GRU (res, L168)	212	0.52	[0.21, 0.94]	58%	0.35 .. 0.60
TCN (res, L168)	GRU (res, L168)	191	0.31	[0.07, 0.60]	57%	0.20 .. 0.36

The residual GRU is indistinguishable from naive-168h and better on only 43% of the days. The residual TCN is better than the residual GRU by 0.52 kW on average, with an interval from 0.21 to 0.94 kW. It stays above zero, so the difference is probably real. The TCN wins on 58% of days. This is the one architecture effect that survives the paired test. It is smaller on the 191 days without June (0.31 kW, interval 0.07 to 0.60 kW), but still above zero. Against naive-168h the TCN is better by 0.46 kW on average, with an interval from 0.02 to 1.07 kW. The lower end is almost zero, so this gain is uncertain. The TCN is better on fewer than half of the days (47%). The gain is concentrated. The median daily difference is −0.06 kW, and the mean gain falls from 0.46 to 0.12 kW when the ten days with the largest gains are removed. These days lie mostly between 12 and 21 June 2014. On 12 to 14 June the total load was 13 to 15% above that of the previous week. A weather event is plausible, but public holidays do not fit because they would lower the load. I did not check the cause. The data for 26 December are an exact copy of 19 December in 23 of 24 hours, so naive-168h is almost perfect on that day. Without it the TCN gain over naive-168h is 0.50 kW (interval 0.08 to 1.08 kW). I keep the day in all tables,

because the mean tables come from runs without saved forecasts. It changes the mean MAE by 0.07 kW for naive-168h and by 0.03 kW for the TCN.

3.3 Input length and receptive field

Figure 3 shows the residual test MAE by input length. The plain models show no systematic benefit either (sweep not shown).

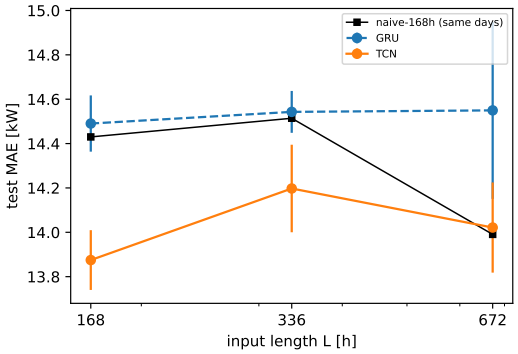


Figure 3: Test MAE of the residual models by input length, with the standard deviation over 5 seeds as error bars. The points for different L are scored on different test days (Section 2). The black line is naive-168h on the same days as the models at each L.

Longer inputs do not help. On common days the residual GRU at $L = 672$ is worse than at $L = 168$ (-0.42 kW, CI $[-0.71, -0.03]$). It is also worse than naive-168h (-0.56, $[-0.94, -0.07]$). The residual TCN at $L = 336$ is slightly worse than at $L = 168$ (-0.20, $[-0.38, -0.04]$). At $L = 672$ the TCN is level with naive-168h on the same days (14.02 against 13.99 kW) and the GRU is worse (14.55 kW). I therefore do not claim that a longer input improves the models. One week of history is the best or equal-best input length.

I also varied the receptive field of the residual TCN at $L = 672$ (Figure 4) with $n = 3, \ldots, 8$ blocks, that is 29, 61, 125, 253, 509 and 1021 h. The MAE is 14.47, 14.38, 14.37, **14.00**, 14.02 and 14.03 kW (standard deviation 0.08–0.31). It stays at about 14.4 kW while the receptive field is shorter than one week. This is 0.4 kW worse than naive-168h on the same days (13.99 kW). It drops by about 0.37 kW once the receptive field exceeds 168 h, matches naive-168h and then does not improve further. This sweep shows when the network stops harming its anchor and not when it adds value, because the TCN does not beat the anchor at $L = 672$. I did not run it at $L = 168$, where the TCN does beat the anchor. Deeper networks have more parameters (33k to 77k), but the plateau from $n = 6$ to 8 shows that capacity alone does not explain the step. The standard deviations of neighbouring points overlap and I ran no paired test, so the step rests on seed means.

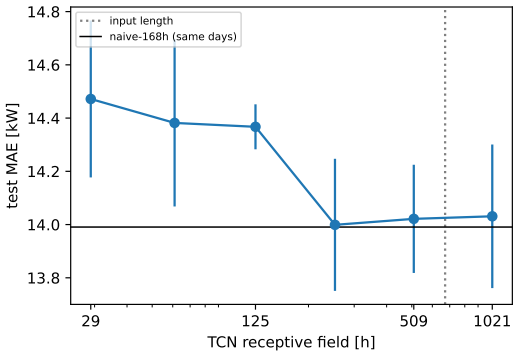


Figure 4: Test MAE of the residual TCN against its receptive field ($L = 672$). The dotted vertical line marks the input length and the horizontal line naive-168h on the same 191 test days.

3.4 A four-week average is at least as good as the learned models

The four-week mean (mean-4w) averages the same 24 hours over the last four weeks. On the 191 days that all methods cover its MAE is 13.23 kW. This is 0.76 kW better than repeating last week (Table 3). Mean-2w

and median-4w are similar (0.49 and 0.69 kW better than last week). Averaging reduces the influence of any single unusual week. The unusual days of June are not in this set, because it starts on 24 June. On these days the TCN's gain over naive-168h is only 0.17 kW (13.99 against 13.82 kW). None of the learned models beats mean-4w. The residual TCN is 0.6 kW worse on average. The interval starts at −0.01 kW, so a tie is possible. The residual GRU is 0.9 kW worse, and the interval is clearly above zero.

Table 3: Test MAE on the 191 common days and the extra MAE compared with mean-4w, with its 95% interval. A positive value means worse than mean-4w. "Residual on mean-4w" means that the network corrects mean-4w instead of last week's load.

method	MAE [kW]	extra MAE vs. mean-4w [kW]
mean-4w	13.23	–
naive-168h (last week)	13.99	+0.76 [0.14, 1.42]
TCN, residual ($L = 168$)	13.82	+0.59 [−0.01, 1.17]
GRU, residual ($L = 168$)	14.13	+0.90 [0.28, 1.56]
TCN, residual on mean-4w ($L = 672$)	13.70	+0.47 [0.00, 0.83]
GRU, residual on mean-4w ($L = 672$)	13.90	+0.67 [0.02, 1.15]

I also trained networks to correct mean-4w itself (last two rows). They are still not better than the mean they start from, so the learned correction adds error. The result does not depend on the metric. Mean-4w also has the lower normalised MSE, which is the training metric (0.1195 against 0.1210 for the TCN and 0.1253 for the GRU), and the lower mean WAPE over the clients (8.99% against 9.36% and 9.42%). On the validation days the same networks had a 7% lower normalised MSE than mean-4w (0.116 against 0.125, 80 days). This points to overfitting to the validation period or to a difference between the two periods, but the evidence is weak, because early stopping picks the best validation epoch. Results also vary strongly between seeds (up to 1.5 kW for one TCN seed).

3.5 Per client

Table 4: Relative error (WAPE, %) per client, that is client MAE divided by its mean load. All columns use the same 191 test days. The lowest value per row is in bold.

client	mean load [kW]	naive-24h	naive-168h	mean-4w	GRU (res)	TCN (res)
MT_156	88	24.5	14.1	**12.0**	14.5	14.1
MT_159	45	48.1	28.8	**26.8**	29.7	32.1
MT_169	42	5.9	6.5	**5.8**	6.5	6.3
MT_232	81	**6.2**	7.4	6.6	7.4	7.3
MT_242	179	**6.9**	8.2	7.9	8.6	8.5
MT_248	156	**5.1**	6.1	5.5	6.1	5.7
MT_266	267	**6.3**	7.0	6.7	7.0	7.0
MT_281	110	**4.7**	5.6	5.5	5.5	5.5
MT_302	117	**5.1**	6.4	7.0	6.4	6.0
MT_307	766	**6.0**	6.4	6.1	6.4	6.1

Two clients, MT_156 and MT_159, are hard to forecast for every method. Their relative error is between 12% and 48%, against about 5–9% for the others. Repeating yesterday has a lower relative error than every learned model on 8 of the 10 clients. It is much worse on those two clients, so its overall MAE is no better than repeating last week. The residual models are tied to last week's load. For clients with little weekly pattern yesterday is the better starting point. The networks do not fully make up for this. The overall MAE in kW is dominated by the large clients. MT_307 alone contributes about a third.

4 Discussion

Why did the TCN not clearly beat the GRU when trained directly? Bai et al. [1] report that generic convolutional networks match or beat recurrent ones on sequence tasks. In my direct setup the TCN (16.58 kW) and the GRU (16.49 kW) are within seed noise, and only the LSTM is clearly worse (17.87 kW). I see three possible reasons and tested none of them separately. First, my task is a day-ahead forecast of a strongly seasonal signal and not one of their benchmark tasks. Second, my models are small (about 68k parameters) and tuned little. Third, the direct networks spend capacity on something the baseline gets for free, as the next paragraph argues. Lai et al. [2] expect recurrent models to struggle with long periodic patterns. The GRU indeed does not profit from a window longer than one week, and neither does the TCN (plain-model sweep, not shown).

Why do direct networks lose to "repeat last week", and why does the residual help? Naive-168h is a strong baseline, because load is dominated by weekly repetition. A direct network has to reconstruct last week's values from normalised inputs. The information passes through a bottleneck before the output layer (the last hidden state or time step, 38–72 units). This costs accuracy on the part of the signal that is easiest to predict. The residual formulation hands that part over directly. The size of the effect fits this explanation (2.0 kW for the GRU, 2.7 kW for the TCN). It does not prove it. A residual target also has a lower variance and may be easier to optimise. The direct networks must also infer the time of day, because they get no calendar input and are trained on windows that start at every hour but tested at midnight. I did not test these alternatives.

Why is the TCN better than the GRU only with the residual? With the residual the TCN is reliably better (0.52 kW, CI [0.21, 0.94]). The longer window is not the reason, since $L = 168$ is as good as longer inputs. The receptive-field experiment points to what matters. A TCN with a receptive field shorter than one week is worse than the anchor it starts from, so its correction adds error. The error drops to the level of the anchor once the receptive field covers a week. The sweep ran at $L = 672$, so it shows when the network stops harming the anchor and not when it adds value. A plausible reading is that dilated convolutions can look directly at fixed lags such as 24 and 168 hours back. They can then estimate how last week's values should be corrected. The GRU has to carry the same information through up to 168 time steps in a state of 72 units. The alternatives are the convolutional inductive bias in general and differences in capacity or optimisation. I did not separate them. In the direct setup the advantage is absent, which fits the idea that it only shows when the task is a small correction and not the seasonal profile.

How does this relate to LSTNet? Lai et al. [2] add a linear autoregressive component to the network output and a recurrent-skip layer that connects time steps one period apart. My residual anchor is closest to the autoregressive component. Both are linear shortcuts added to the network output. Mine is fixed (last week or the four-week mean) and theirs is learned. The skip layer is a different mechanism, because it changes the recurrent network itself. As I understand it, it targets the long periodic pattern that the GRU struggles with here. I did not implement or test it, so this compares ideas and not results.

What do the networks add? Little, in this setting. Past load is largely explained by its weekly profile and by the multi-week average. The corrections the networks learn do not reliably beat a four-week average. This holds even when the networks are trained directly on it. It holds for the tested configurations and period and is not a general claim that neural networks cannot improve on seasonal averages. It does suggest that a comparison of architectures on forecasting data needs a strong seasonal baseline.

Limitations. I used only 10 clients and one test period of about 200 days, so the conclusions may depend on the period. Each architecture has one hyperparameter setting, with capacity matched by parameter count. The models see past load only. They get no calendar, holiday or weather inputs, which probably handicaps the direct networks. Training windows start at every hour, while test windows start at midnight. This may cost the direct models accuracy. I tested neither point. The networks are trained on normalised MSE, which weights all clients equally, while the MAE in kW is dominated by one large client. Mean-4w wins in both (Section 3.4). The residual formulation and the multi-week baselines were developed after I had seen test results, so the conclusions are exploratory. I also did not trace the concentration of the TCN's gain in June 2014 to a cause. A scan of all 1096 days found no other copied day across clients. Some single-client days repeat an earlier day exactly (mostly MT_156 in 2012 and MT_302 in 2013), none in the test period. I did not clean them.

Conclusion. Directly trained LSTM, GRU and TCN forecasters lose to "repeat last week" for 24-hour-ahead load of 10 clients. Predicting a correction to last week's load removes the gap. It also makes the TCN reliably better than the GRU. The TCN's advantage over the seasonal baseline is nevertheless small and rests on a few days. A four-week average is at least as accurate as every learned model I trained. This includes the networks trained to correct the average. For this task a strong seasonal baseline and paired evaluation over test days matter more than the choice of architecture.

References

[1] S. Bai, J. Z. Kolter, and V. Koltun, "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling," arXiv:1803.01271, 2018.

[2] G. Lai, W.-C. Chang, Y. Yang, and H. Liu, "Modeling long- and short-term temporal patterns with deep neural networks," in *Proc. 41st Int. ACM SIGIR Conf. Research and Development in Information Retrieval*, 2018.

[3] A. Trindade, "ElectricityLoadDiagrams20112014," UCI Machine Learning Repository, 2015. [Online]. Available: `https://doi.org/10.24432/C58C86`